

Memoria técnica del proyecto

Aplicación de gestión de aulas y reservas en Java

Jose Luis López Programación

1 º DAW 2025/2026

Alejandro Gutiérrez Villalba

Índice

INTRODUCCIÓN	3
Contexto del proyecto	3
Objetivos del proyecto	3
TECNOLOGÍAS UTILIZADAS	4
Lenguaje de programación Java	4
Base de datos: MySQL Workbench y MariaDB	4
Librería JDBC (Java Database Connectivity)	4
XML y XPath	5
Patrón DAO (Data Access Object)	5
Modelo de base de datos.....	5
Decisiones de diseño en la BBDD:	6
DESARROLLO DEL PROYECTO	7
Fases del desarrollo	7
DIFICULTADES ENCONTRADAS Y SOLUCIONES	8
FUNCIONAMIENTO DEL SISTEMA.....	10
Manual de Usuario	10
APRENDIZAJES Y CONCLUSIONES.....	10
Bibliografía y Webgrafía	12

INTRODUCCIÓN

Contexto del proyecto

Este proyecto surge como respuesta a una simulación de una necesidad real en centros educativos, la gestión eficiente de espacios de estudio y el control de material informático prestable. En muchos institutos, escuelas, universidades, los alumnos necesitan reservar puestos en aulas o bibliotecas para estudiar, y a menudo necesitan dispositivos como portátiles o tablets para realizar sus trabajos y estudios.

El problema principal que identifiqué es la falta de un sistema centralizado que permita controlar quién reserva qué puesto y cuándo, gestionar los préstamos de dispositivos, llevar un registro de retrasos en las devoluciones, y aplicar sanciones de forma automática a usuarios que no cumplen con los plazos

Objetivos del proyecto

El objetivo principal era desarrollar una aplicación de consola en Java que permitiera gestionar de forma integral. Usuarios del sistema (alumnos y profesores) con operaciones de alta, baja y modificación, reservas de puestos de estudio en diferentes aulas y salas, préstamos de dispositivos digitales con control de fechas, detección automática de retrasos y aplicación de sanciones, generación de informes en formato XML con estadísticas de uso, y todo esto debía hacerse siguiendo los principios de la Programación Orientada a Objetos, utilizando una base de datos relacional, y usando los conocimientos para configurar una máquina virtual y conectarlo todo.

El proyecto cubre la gestión completa de usuarios, reservas y préstamos, persistencia en base de datos con Workbench, Exportación de datos a XML, consultas sobre archivos XML usando XPath, e Interfaz de consola clara e intuitiva

TECNOLOGÍAS UTILIZADAS

Lenguaje de programación Java

Elegí Java como lenguaje principal porque principalmente es el lenguaje que estamos aprendiendo en el módulo de Programación, tiene soporte para programación orientada a objetos, cuenta con librerías para trabajar con bases de datos (JDBC) y XML

Base de datos: MySQL Workbench y MariaDB

Opté por MySQL Workbench porque fue el que vimos en el momento de plantear la base de datos, es un sistema de gestión de bases de datos relacional muy utilizado en el mundo profesional, es gratuito y tiene buena integración con Java a través de JDBC. En mi caso específico, utilicé MariaDB en el puerto 3307 de mi máquina virtual.

Unas ventajas de usar una base de datos relacional son que los datos persisten aunque cierre la aplicación, puedo hacer consultas complejas con SQL, y garantiza la integridad de los datos mediante claves foráneas.

Librería JDBC (Java Database Connectivity)

JDBC es la tecnología de Java que permite conectar con bases de datos. En mi proyecto utilicé el driver mariadb-java-client e implementé el DAO (Data Access Object) para separar la lógica de base de datos.

XML y XPath

Para los informes y estadísticas utilicé: DOM (Document Object Model) para crear y escribir archivos XML desde Java, XPath para consultar y extraer información de los archivos XML generados. Estas tecnologías son estándar en Java y no requieren librerías externas.

Patrón DAO (Data Access Object)

Para cada entidad importante creé un DAO que gestiona todas las operaciones con la base de datos:

UsuarioDAO: Añadir, listar, modificar, dar de baja usuarios

PuestoDAO: Ver puestos por sala, buscar por ID, cambiar estado

ReservaDAO: Crear, cancelar y consultar reservas

DispositivoDAO: Ver disponibles, buscar, cambiar estado

PrestamoDAO: Registrar préstamos, devoluciones, ver activos/retrasados

Modelo de base de datos

Diseñé un modelo relacional normalizado con 6 tablas, las tablas principales son:

1. Usuarios: Almacena tanto alumnos como profesores en una sola tabla
2. Puestos: Los puestos de estudio disponibles en diferentes aulas
3. Dispositivos: Ordenadores y tablets prestables
4. Reservas: Relaciona usuarios con puestos (incluye fecha y hora)
5. Prestamos: Relaciona usuarios con dispositivos (incluye fechas)

6. Sanciones: Registro de sanciones por retrasos

Decisiones de diseño en la BBDD:

Tabla única para usuarios: En lugar de crear tablas separadas para alumnos y profesores, utilicé una sola tabla con un campo que indica si es alumno o profesor. Los campos específicos (curso para alumnos o materia para profesores) pueden ser NULL.

Tabla única para dispositivos: Mismo modo que con usuarios. Un campo que distingue entre Ordenador y Tablet.

Relaciones con claves foráneas: Todas las tablas están correctamente relacionadas:

reservas.id_usuario --- usuarios.id

reservas.id_puesto --- puestos.id

prestamos.id_usuario --- usuarios.id

prestamos.id_dispositivo --- dispositivos.id

sanciones.id_usuario --- usuarios.id

sanciones.id_prestamo --- prestamos.id

En cuanto a restricciones:

Correo y DNI únicos en usuarios (no puede haber duplicados), no puede haber dos puestos con el mismo número en la misma aula, no puede haber dos reservas del mismo puesto en la misma fecha/hora.

DESARROLLO DEL PROYECTO

Fases del desarrollo

El proyecto se desarrolló en varias etapas claramente diferenciadas:

1: Esquema y Diseño de clases (POO) y diagrama ER:

Lo primero que hice fue diseñar las clases del modelo. Empecé con las clases básicas:

Usuario (abstracta) con sus hijos Alumno y Profesor, Dispositivo (abstracta) con sus hijos Ordenador y Tablet, y Puesto, Reserva y préstamo

2: Diseño de la base de datos

Creé los scripts SQL para crear las 6 tablas con sus campos y tipos de datos, establecer las claves primarias y foráneas, añadir restricciones de integridad e insertar datos de prueba.

3: Conexión Java-MySQL (JDBC)

Implementé la clase ConexionBD.java que gestiona la conexión con la base de datos, configuré la URL de conexión adaptada a mi entorno:

```
jdbc:mariadb://127.0.0.1:3307/gestion_aulas?user=guti&password=guti
```

4: Implementación de DAOs

Esta fue la fase más larga. Creé un DAO por cada entidad principal.

5: Menús de consola

Desarrollé el sistema de menús en Main.java:

Menú principal con 4 opciones y submenús para cada sección (Usuarios, Reservas, Préstamos, Informes)

6: Control de retrasos

Al registrar una devolución, el sistema calcula automáticamente si hay retraso. Si hay retraso, cambia el estado del usuario a "sancionado", registra la sanción en la tabla correspondiente y los usuarios sancionados no pueden hacer más préstamos ni reservas.

Validaciones: No se puede reservar un puesto ya ocupado, no se puede prestar un dispositivo que no está disponible, los usuarios de baja o sancionados no pueden operar.

7: Exportación a XML y consultas XPath

ExportadorXML.java: lee datos de la base de datos usando SQL, construye un documento XML, y guarda el XML en archivos (reservas.xml, prestamos.xml)

LectorXML.java: lee los archivos XML generados, usa XPath para hacer consultas complejas y genera estadísticas (total de reservas, usuarios con más reservas, dispositivos más prestados, etc.)

8: Documentación

Manual de usuario: Guía paso a paso para cualquier persona que use el sistema y lo pueda entender correctamente.

DIFICULTADES ENCONTRADAS Y SOLUCIONES

Problema: Relación entre clases Java y tablas de BD: al principio no tenía claro cómo relacionar las clases Java (Prestamo tiene un objeto Usuario) con la base de datos (prestamos tiene un id_usuario). En la BD solo guardas IDs (números).

Problema: Cálculo de días de retraso

Necesitaba calcular la diferencia en días entre dos fechas para saber si hay retraso.

Solución, usé el método `toEpochDay()` que convierte una fecha a un número. La diferencia entre dos números es simplemente una resta:

```
long dias = fechaReal.toEpochDay() - fechaPrevista.toEpochDay();
```

Es más simple de entender y no requiere conocimientos avanzados.

Problema: Warnings de SLF4J en consola

Al ejecutar la aplicación aparecían warnings molestos sobre SLF4J, investigué y descubrí que son solo warnings, no errores, el programa funciona perfectamente con ellos. Para eliminarlos habría que añadir librerías adicionales (`slf4j-simple.jar`), decidí dejarlos así porque no afectan la funcionalidad.

En un entorno profesional se añadirían las librerías, pero para este proyecto no era necesario.

Problema: Diseño de las tablas

Dudaba si crear tablas separadas para alumnos y profesores, o una sola tabla unificada.

Después de investigar sobre normalización de bases de datos, opté por una sola tabla usuarios con un campo, hace las consultas más simples, menos código, más fácil de mantener.

Problema: Formato de fechas y horas

Las fechas se guardaban en la BD en un formato y las necesitaba mostrar en otro.

Usé los métodos de conversión de JDBC: Java facilita mucho este trabajo con las clases `LocalDate` y `LocalTime`.

`Date.valueOf(LocalDate)` para convertir Java a SQL

`rs.getDate("fecha").toLocalDate()` para convertir SQL a Java

Problema: Testing y depuración

Al principio era difícil saber si el código funcionaba sin tener datos en la BD, creé
qurys de datos de prueba.

FUNCIONAMIENTO DEL SISTEMA

Se adjunta toda la información para su uso en el manual de usuario adjunto.

Manual de Usuario



MANUAL_USUARIO.p
df

APRENDIZAJES Y CONCLUSIONES

Conocimientos técnicos adquiridos: durante el desarrollo de este proyecto he aprendido
que la programación Orientada a Objetos se basa en el diseño de jerarquías de
clases, Uso práctico de herencia y polimorfismo, y la importancia de la
encapsulación.

Trabajo con bases de datos relacionales: diseño de esquemas normalizados, uso de
claves primarias y foráneas, Consultas SQL

Manejo de XML: creación de documentos XML, consultas con XPath, exportación de
datos

Gestión de fechas en Java: (`LocalDate`, `LocalTime`)

Habilidades desarrolladas: más allá de lo técnico, este proyecto me ha ayudado a planificar y dividir un proyecto grande en tareas más manejables.

La resolución de problemas al enfrentarme a errores y buscar soluciones. Escribir código claro y documentarlo adecuadamente y la persistencia de no rendirme ante problemas complejos.

Posibles mejoras futuras para ampliar el proyecto en el futuro: Sistema de autenticación, login con usuario y contraseña, diferentes niveles de permisos (admin vs usuario normal), notificaciones automáticas, envío de emails recordando devoluciones, alertas cuando un dispositivo está próximo a vencerse.

Conclusión: este proyecto ha sido un reto estresante, importante pero muy enriquecedor. He podido aplicar de forma práctica todos los conceptos vistos en clase y buscados en internet. Programación Orientada a Objetos, bases de datos relacionales, manejo de ficheros (XML), etc...

Lo que más costó ha sido entender cómo todas las tecnologías se conectan entre sí: Java con MySQL a través de JDBC, los DAOs que traducen entre objetos y tablas, el XML que permite exportar datos, y XPath que permite consultarlos.

He comprobado que la programación no es solo escribir código, sino pensar bien la estructura, anticipar problemas (si se puede), validar datos, documentar adecuadamente y al día, y mantener el código limpio y organizado.

El resultado es una aplicación funcional que resuelve un problema real y que podría ser utilizada en un centro educativo. Estoy satisfecho con el trabajo realizado y con todo lo que he aprendido en el proceso.

Bibliografía y Webgrafía

Lenguaje Java

Oracle Java Documentation: <https://docs.oracle.com/en/java/>

Java SE 11 Documentation: <https://docs.oracle.com/en/java/javase/11/>

Base de Datos

MySQL Official Documentation: <https://dev.mysql.com/doc/>

MySQL Workbench Manual: <https://dev.mysql.com/doc/workbench/en/>

JDBC (Java Database Connectivity)

MariaDB Connector/J: <https://mariadb.com/kb/en/about-mariadb-connector-j/>

XML y XPath

XPath Tutorial W3Schools: https://www.w3schools.com/xml/xpath_intro.asp

Programación Orientada a Objetos

Oracle Java OOP Concepts: <https://docs.oracle.com/javase/tutorial/java/concepts/>

